

Algoritmi Avansați

Seminarul 1

1 Introducere

Ce este o problemă (computațională)?

O **problemă computațională** [1] este o problemă la care putem determina răspunsul folosind un algoritm¹.

- **Exemplu de problemă computațională:** Care sunt factorii primi ai numărului natural n ?
- **Exemplu de problemă care nu se poate rezolva folosind algoritmi:** Ce pereche de pantaloni să port astăzi?²

Ce tipuri de probleme computaționale există?

- **Probleme de decizie** [5]: Răspunsul este de tipul adevărat/fals (posibil/imposibil, există/nu există etc.)
- **Probleme de optimizare** [6]: Răspunsul este o *soluție* (un număr, un șir de numere, un text etc.) care este “cea mai bună” dintr-un anumit punct de vedere (consumă cele mai puține resurse, produce cel mai mare profit, acoperă cele mai multe noduri etc.) față de toate celelalte soluții posibile.

¹Un *algoritm* [2, 3] este o serie de pași bine-definiți care îți permit să obții un rezultat plecând de la niște date de intrare. În comparație, “să ghicești răspunsul corect” nu e tocmai un algoritm. Dar poate fi o euristică [4].

²Dacă ai un număr finit de opțiuni (pantaloni din care să alegi) și asociezi fiecărei perechi câte un *scor* (număr) care să indice deziderabilitatea acesteia, atunci această problemă se poate reduce la o problemă computațională, aceea de a găsi perechea de pantaloni cu scorul maxim.

- **Probleme de numărare [7]:** Răspunsul este *numărul total de soluții* care satisfac o anumită proprietate. În loc să căutăm *dacă* există o soluție sau care este cea mai bună, suntem interesați de *câte* astfel de soluții există în total

2 Exerciții

Cunoștințe generale

1. Dați exemplu de **2 probleme** pe care le întâlniți în viața de zi cu zi care se pot interpreta ca probleme computaționale (de exemplu, cum alegeți traseul pe care să vii la facultate).
2. Dați exemplu de **2 probleme de decizie** și **2 probleme de optimizare** pe care le-ați întâlnit în practică sau de care ați auzit.

P versus NP

3. În cele ce urmează, să presupunem că avem o **listă** (un vector) de n **numere întregi**.
 1. Propuneți un algoritm care să determine **cel mai mare element** (în **valoare absolută**) din listă. Ce complexitate de timp/memorie are algoritmul propus?
 2. Pentru un număr întreg dat S , propuneți un algoritm care să determine **toate perechile de numere** din lista inițială **care adunate dau S** . Puteți găsi un algoritm cu complexitate de timp, în cel mai rău caz, mai bună decât $\mathcal{O}(n^2)$?
 3. (3SUM [8]) Propuneți un algoritm care să determine dacă în lista inițială există **trei numere** care **adunate să aibă suma 0**. Credeți că puteți un algoritm determinist care să rezolve problema în mai puțin de n^2 pași? Dar dacă ați aborda problema în mod nedeterminist?

Metoda greedy

Un algoritm de tip *greedy* va încerca la fiecare pas să ia alegerea optimă, care îi dă un „câștig” maxim imediat (optim local). Mai multe informații despre principiile care stau la baza metodei greedy se pot găsi la [9] sau în [3].

4 (Problema spectacolelor / intervalelor). [10] Suntem administratorii unui spital și vrem să ne asigurăm că avem tot timpul cel puțin un medic prezent la camera de gardă într-un anumit interval de timp. Presupunem că avem la dispoziție n medici, care ar fi dispuși să stea de gardă fiecare între anumite ore. Vrem să asignăm câți mai puțini medici, ca ceilalți să se poată ocupa de alte urgențe.

Problema formalizată: Fiind dat un interval $[a, b]$ și o mulțime de intervale $[a_1, b_1], [a_2, b_2], \dots, [a_n, b_n]$, vrem să alegem un număr *minim* dintre acestea astfel încât, reunite, să includă intervalul inițial $[a, b]$.

1. Aceasta este o problemă de **decizie** sau de **optimizare**? Dar dacă ne-ar interesa doar să vedem dacă putem acoperi intervalul dat cu intervalele disponibile?
2. Propuneți un algoritm care **să determine dacă** măcar putem acoperi intervalul dat cu intervalele disponibile.
3. Propuneți un algoritm care să rezolve problema inițială. **Demonstrați** că algoritmul propus obține soluția optimă (adică că nu poate exista o altă soluție, diferită de cea obținută prin metoda voastră, care să folosească mai puține intervale).

3 Probleme

Problema rucsacului

Descrierea problemei. Se dă o mulțime de obiecte $O = \{o_1, o_2, \dots, o_n\}$. Pentru fiecare obiect o_i știm cât este **valoarea** obiectului (pe care o vom nota val_i), respectiv **greutatea** obiectului (pe care o vom nota g_i).

Scopul este să găsim o submulțime de obiecte $S \subseteq O$ astfel încât suma greutateilor obiectelor din S să nu depășească o capacitate C a unui rucsac (fixată de la început), iar valoarea totală a acestora să fie maximă.

Formularea matematică. Să se găsească $S \subseteq O$ care să maximizeze

$$\sum_{o_i \in S} val_i$$

cu condiția că

$$\sum_{o_i \in S} g_i \leq C$$

Observație. Toate submulțimile S care respectă condiția de mai sus se vor numi **soluții candidat**. Dintre acestea, soluția (sau soluțiile) care maximizează valoarea totală se va numi **soluția optimă**.

Varianta fracționară [11]

Elementele din S pot fi fracțiuni din obiecte; cu alte cuvinte, pentru fiecare obiect o_i reținem și procentul din el pe care îl luăm în considerare, $p_i \in [0, 1]$. Fiecare obiect din soluție contribuie la valoarea totală cu $p_i \cdot val_i$, respectiv este contorizat spre greutatea totală cu $p_i \cdot g_i$.

Formularea matematică devine: să se găsească $S \subseteq O$ care să maximizeze

$$\sum_{o_i \in O} p_i \cdot val_i$$

cu condiția că

$$\sum_{o_i \in O} p_i \cdot g_i \leq C$$

Complexitatea pe care ar trebui să o aibă soluția voastră: $\mathcal{O}(n \log n)$

Varianta discretă [12]

Elementele din S pot fi doar obiecte luate cu totul; cu alte cuvinte, pentru fiecare obiect o_i decidem dacă îl includem sau nu.

Complexitatea pe care ar trebui să o aibă soluția voastră: $\mathcal{O}(n \cdot C)$

Alte probleme pseudo-polinomiale

Submulțime de elemente cu suma dată

Se dă o mulțime de n numere naturale și un număr natural $M < 10000$. Să se determine, dacă există, o submulțime a mulțimii date de sumă egală cu M .

Exemplu. Pentru $n = 6$, mulțimea $\{12, 1, 3, 4, 5, 7\}$ și $M = 14$, o soluție este submulțimea $\{3, 4, 7\}$.

Complexitatea pe care ar trebui să o aibă soluția voastră: $\mathcal{O}(n \cdot M)$

Împărțirea cadourilor

Moș Crăciun a poposit la bradul a doi frați, unde și-a golit sacul. Când s-au trezit, frații au intrat într-o mare dilemă: cum își vor împărți ei cadourile? Știind că fiecare cadou are o valoare cuprinsă între 1 și 100 și că sunt maxim

100 de cadouri, scrieți un program care să determine sumele cadourilor fraților precum și modul de împărțire, astfel încât sumele obținute să fie cât de apropiate posibil.

Exemplu. Pentru 7 cadouri cu valorile $\{28, 7, 11, 8, 9, 7, 27\}$, sumele sunt 48 și 49, o împărțire a cadourilor fiind: $\{28, 11, 9\}$, respectiv $\{7, 8, 7, 27\}$.

Complexitatea pe care ar trebui să o aibă soluția voastră: $\mathcal{O}(n \cdot S)$, unde S reprezintă valoarea totală a cadourilor.

Alte probleme de programare dinamică

Generalizarea problemei spectacolelor

Se dau n activități, pentru fiecare având: **timpul de început**, **timpul de sfârșit** și **profitul** asociat desfășurării activității (adică n intervale de numere reale, fiecare cu o pondere asociată). Să se determine o submulțime de activități compatibile (intervale disjuncte două câte două) care au profitul total maxim. Se vor afișa profitul total și activitățile selectate.

Exemplu. Pentru $n = 4$ și activitățile:

- $A_1 = [1, 3]$ cu profitul $p_1 = 1$
- $A_2 = [2, 6]$ cu profit $p_2 = 8$
- $A_3 = [4, 7]$ cu profit $p_3 = 2$
- $A_4 = [10, 15]$ cu profit $p_4 = 5$

O soluție optimă ar fi formată din activitățile A_2 și A_4 (adică intervalele $[2, 6]$ și $[10, 15]$), cu profitul total 13.

Complexitatea pe care ar trebui să o aibă soluția voastră: încercați să găsiți o soluție în $\mathcal{O}(n \log n)$; dacă nu reușiți, încadrați-vă în cel mult $\mathcal{O}(n^2)$.

Generalizarea problemei planificării activităților

Se dă o listă de n activități. Pentru fiecare activitate cunoaștem **profitul** (p_i), **termenul limită** până la care trebuie efectuată (t_i) și **durata** acesteia (l_i). Trebuie să găsim o submulțime *ordonată* (o sublistă) de activități pentru a obține un profit total maxim.

Exemplu. Pentru $n = 4$ și activitățile:

- A_1 cu $p_1 = 3$, $t_1 = 5$, $l_1 = 3$

- A_2 cu $p_2 = 2, t_2 = 2, l_2 = 1$
- A_3 cu $p_3 = 3, t_3 = 2, l_3 = 2$
- A_4 cu $p_4 = 5, t_4 = 4, l_4 = 3$

O soluție optimă se obține dacă planificăm activitățile în ordinea A_2, A_4 , profitul fiind 7.

Observație. Problema discretă a rucsacului poate fi privită ca un caz particular al acestei probleme (obiectele sunt activități de durată g_i , profit c_i și termen limită C).

Complexitatea pe care ar trebui să o aibă soluția voastră: $\mathcal{O}(n \cdot T + n \log n)$, unde $T = \max \{ t_i \}$.

Problema sumei țintă (Target Sum) [13]

Descrierea problemei. Se dă o mulțime de n numere naturale $A = \{a_1, a_2, \dots, a_n\}$ și un număr întreg T (numit țintă). Pentru fiecare număr din mulțime, trebuie să îi atribuim un simbol matematic, fie $+$ (plus), fie $-$ (minus), astfel încât rezultatul operațiilor succesive să fie egal cu T .

Formularea matematică. Să se determine în câte moduri se pot alege coeficienții $s_i \in \{-1, 1\}$ astfel încât să fie satisfăcută egalitatea:

$$\sum_{i=1}^n s_i \cdot a_i = T$$

Observație. Această problemă este o variantă a problemei submulțimii de sumă dată. Dacă notăm cu P suma elementelor cu semnul $+$ și cu N suma celor cu semnul $-$, avem:

1. $P - N = T$
2. $P + N = S$, unde $S = \sum a_i$

Din cele două rezultă $2P = T + S \implies P = \frac{T+S}{2}$. Problema se reduce la a găsi numărul de submulțimi care au suma exact P .

Exemplu. Pentru $n = 5, A = \{1, 1, 1, 1, 1\}$ și $T = 3$, există 5 variante (moduri) de a obține rezultatul (prin alegerea unui singur element care să aibă semnul minus).

Complexitatea pe care ar trebui să o aibă soluția voastră: $\mathcal{O}(n \cdot S)$, unde S este suma elementelor din A . Deoarece depinde de valoarea numerică S , problema este *pseudo-polinomială*.

Referințe

- [1] Wikipedia contributors, *Computational problem*, URL: https://en.wikipedia.org/wiki/Computational_problem.
- [2] Wikipedia contributors, *Algorithm*, URL: <https://en.wikipedia.org/wiki/Algorithm>.
- [3] Thomas H. Cormen et al., *Introduction to Algorithms*, a 3-a ed., MIT Press, Iul. 2009, URL: <https://www.cs.mcgill.ca/~akroitt/math/compsci/Cormen%20Introduction%20to%20Algorithms.pdf>.
- [4] Wikipedia contributors, *Heuristic*, URL: [https://en.wikipedia.org/wiki/Heuristic_\(computer_science\)](https://en.wikipedia.org/wiki/Heuristic_(computer_science)).
- [5] Wikipedia contributors, *Decision problem*, URL: https://en.wikipedia.org/wiki/Decision_problem.
- [6] Wikipedia contributors, *Optimization problem*, URL: https://en.wikipedia.org/wiki/Optimization_problem.
- [7] Wikipedia contributors, *Counting problem*, URL: [https://en.wikipedia.org/wiki/Counting_problem_\(complexity\)](https://en.wikipedia.org/wiki/Counting_problem_(complexity)).
- [8] Wikipedia contributors, *3SUM*, URL: <https://en.wikipedia.org/wiki/3SUM>.
- [9] Karleigh Moore, Jimin Khim și Eli Ross, *Greedy Algorithm*, URL: <https://brilliant.org/wiki/greedy-algorithm/>.
- [10] Platforma CMS-FMI, *Acoperire cu intervale*, URL: <https://cms.fmi.unibuc.ro/problem/intervals>.
- [11] Platforma CMS-FMI, *Problema rucsacului - varianta fracționară*, URL: <https://cms.fmi.unibuc.ro/problem/fractionalknapsack>.
- [12] Platforma CMS-FMI, *Problema rucsacului - varianta discretă*, URL: <https://cms.fmi.unibuc.ro/problem/discreteknapsack>.
- [13] LeetCode, *Target Sum*, URL: <https://leetcode.com/problems/target-sum/description/>.